Unit : 4 PROGRAMS

- Write a Contact Management System using MVC where users can add contacts with names and phone numbers, and view all saved contacts. The Model handles contact data and interactions with the data store, the View includes a form for adding contacts and a display page for listing them, and the Controller processes input from the form, updates the model, and directs users to the appropriate view.

Model – Contact.java

```
package model;

public class Contact {
    private String name;
    private String phone;
    public Contact() {
    }
    public Contact(String name, String phone) {
        this.name = name;
        this.phone = phone;
    }
    public String getName() {
        return name;
    }
    public void setName(String name) {
        this.name = name;
    }


    public String getPhone() {
```

```
        return phone;
    }
    public void setPhone(String phone) {
        this.phone = phone;
    }
}
```

Model – ContactModel.java

```
package model;
import java.util.ArrayList;
import java.util.List;
public class ContactModel {
    private static List<Contact> contacts = new ArrayList<>();
    // Add contact
    public void addContact(String name, String phone) {
        Contact contact = new Contact(name, phone);
        contacts.add(contact);
    }
    // Get all contacts
    public List<Contact> getAllContacts() {
        return contacts;
    }
}
```

View – addContact.jsp

```
<%@ page language="java"
    contentType="text/html; charset=UTF-8"  pageEncoding="UTF-8"%>
```

```
<html>
<head></head>
<body>
   <h2>Contact Management System</h2>
   <h3>Add New Contact</h3>
   <form action="contacts" method="post">
      Name:
      <input type="text" name="name" required>
      <br><br>
      Phone Number:
      <input type="text" name="phone" required>
      <br><br>
      <input type="submit" value="Add Contact">
   </form>    <br>
   <a href="contacts">View All Contacts</a>
</body>
</html>
```

Controller – ContactServlet.java

```
package controller;
import jakarta.servlet.ServletException;
import jakarta.servlet.annotation.WebServlet;
import jakarta.servlet.http.HttpServlet;
import jakarta.servlet.http.HttpServletRequest;
import jakarta.servlet.http.HttpServletResponse;
```

```
import java.io.IOException;
import model.ContactModel;
@WebServlet("/contacts")
public class ContactServlet extends HttpServlet {
   private ContactModel contactModel;
   @Override
   public void init() {
      contactModel = new ContactModel();
   }
   // Handle Add Contact
   @Override
   protected void doPost(HttpServletRequest request,
                         HttpServletResponse response)
         throws ServletException, IOException {

      String name = request.getParameter("name");
      String phone = request.getParameter("phone");
      contactModel.addContact(name, phone);
      response.sendRedirect("contacts");
   }
   // Display all contacts
   @Override
   protected void doGet(HttpServletRequest request,
                        HttpServletResponse response)
         throws ServletException, IOException {
```

```
        request.setAttribute(
                "contacts",
                contactModel.getAllContacts()
        );
        request.getRequestDispatcher("contacts.jsp")
                .forward(request, response);
    }
}
```

View – contacts.jsp

```
<%@ page language="java"
    contentType="text/html; charset=UTF-8"
    pageEncoding="UTF-8"%>
<%@ taglib prefix="c" uri="jakarta.tags.core" %>
<html>
<head>
    <title>All Contacts</title>
</head>
<body>
    <h2>Contact Management System</h2>
    <h3>All Contacts</h3>
    <table border="1" cellpadding="10">
        <tr>
            <th>Name</th>
            <th>Phone Number</th>
        </tr>
```

```
        <c:forEach var="contact" items="${contacts}">
            <tr>
                <td>
                    <c:out value="${contact.name}"/>
                </td>
                <td>
                    <c:out value="${contact.phone}"/>
                </td>
            </tr>
        </c:forEach>
    </table>
    <br>
    <a href="addContact.jsp">Add New Contact</a>
</body>
</html>
```

- Create an entity class called Employee with attributes empno, empnm and department. Use Hibernate to insert an Employee record in the database, and then retrieve and display all employees from the database.

  Create MySQL Database

  pom.xml

  Add Hibernate and MySQL dependencies:

```
<dependencies>
 <!-- Hibernate -->
<dependency>
        <groupId>org.hibernate.orm</groupId>
        <artifactId>hibernate-core</artifactId>
```

```
            <version>6.6.1.Final</version>
        </dependency>
        <!-- MySQL JDBC Driver -->
        <dependency>
            <groupId>com.mysql</groupId>
             <artifactId>mysql-connector-j</artifactId>
            <version>9.0.0</version>
        </dependency>
       </dependencies>
```

Employee.java

src/main/java/entity/Employee.java

Create the entity class inside the entity package:

```
package entity;
import jakarta.persistence.Entity;
import jakarta.persistence.Id;
import jakarta.persistence.Table;
@Entity
@Table(name = "employee")
public class Employee {
   @Id
   private int empno;
   private String empnm;
   private String department;
   // Default constructor
   public Employee() {
   }
```

```
    // Parameterized constructor
    public Employee(int empno, String empnm, String department) {
        this.empno = empno;
        this.empnm = empnm;
        this.department = department;
    }
    // Getters and Setters
    public int getEmpno() {
        return empno;
    }
    public void setEmpno(int empno) {
        this.empno = empno;
    }
    public String getEmpnm() {
        return empnm;
    }
    public void setEmpnm(String empnm) {
        this.empnm = empnm;
    }
    public String getDepartment() {
        return department;
    }
    public void setDepartment(String department) {
        this.department = department;
    }
}
```

hibernate.cfg.xml

src/main/resources/hibernate.cfg.xml

```
<?xml version="1.0" encoding="UTF-8"?>
<!DOCTYPE hibernate-configuration PUBLIC
      "-//Hibernate/Hibernate Configuration DTD 3.0//EN"
      "https://hibernate.org/dtd/hibernate-configuration-3.0.dtd">
<hibernate-configuration>
   <session-factory>
      <!-- MySQL Connection -->
      <property name="hibernate.connection.driver_class">
         com.mysql.cj.jdbc.Driver
      </property>
      <property name="hibernate.connection.url">
         jdbc:mysql://localhost:3306/company
      </property>
      <property name="hibernate.connection.username">
         root
      </property>
      <property name="hibernate.connection.password">
         root
      </property>
      <!-- Hibernate Settings -->
      <property name="hibernate.dialect">
         org.hibernate.dialect.MySQLDialect
      </property>
      <property name="hibernate.hbm2ddl.auto">
```

```
        update
    </property>
    <property name="hibernate.show_sql">
        true
    </property>
    <!-- Entity Mapping -->
    <mapping class="entity.Employee"/>
  </session-factory>
</hibernate-configuration>
```

Main.java

```
package main;
import entity.Employee;
import java.util.List;
import org.hibernate.Session;
import org.hibernate.SessionFactory;
import org.hibernate.cfg.Configuration;
public class Main {
  public static void main(String[] args) {
    // Create SessionFactory
    SessionFactory factory = new Configuration()
        .configure("hibernate.cfg.xml")
        .buildSessionFactory();
    Session session = factory.openSession();
    try {
      Employee emp = new Employee(
          101,
```

```
            "Hello",
            "Computer"
        );
        session.beginTransaction();
        session.persist(emp);
        session.getTransaction().commit();
        System.out.println("Employee inserted successfully!");
        session.beginTransaction();
        List<Employee> employees =
            session.createQuery(
                "from Employee",
                Employee.class
            ).getResultList();
        session.getTransaction().commit();
        // Display employees
        System.out.println("\n--- Employee Details ---");
        for (Employee e : employees) {
            System.out.println("Employee No: "
                + e.getEmpno());
            System.out.println("Employee Name: "
                + e.getEmpnm());
            System.out.println("Department: "
                + e.getDepartment());
    } finally {
        session.close();
        factory.close();
```

```
    }
  }
}
```